

Python – Pandas

What is Pandas?

Pandas is a **Python library** used for **working with data sets**. It provides powerful tools to **analyze, clean, explore, and manipulate data** efficiently. Because of its simplicity and performance, Pandas is widely used in **data science, data analysis, and machine learning**.

The name “**Pandas**” comes from:

- **Panel Data** (a type of structured data)
- **Python Data Analysis**

Pandas was **created by Wes McKinney in 2008**, and since then it has become one of the most important libraries in the Python data ecosystem.

Example:

Using Pandas, you can easily load a CSV file containing student marks and analyze their average, highest score, or missing values.

Why Use Pandas?

Pandas is especially useful when dealing with **large and messy data sets**.

- Pandas allows us to:
 - Analyze **big data**
 - Draw conclusions using **statistical methods**
 - Make data **meaningful and usable**
- One of the biggest advantages of Pandas is **data cleaning**:
 - Removes incorrect data
 - Handles missing or empty values
 - Makes data readable and relevant

Why relevant data matters:

In data science, incorrect or irrelevant data can lead to **wrong conclusions**, even if the analysis code is correct.

What Can Pandas Do?

Pandas helps answer important questions about data, such as:

- Is there a **correlation** between two or more columns?
- What is the **average (mean)** value?
- What is the **maximum** value?
- What is the **minimum** value?

Data Cleaning with Pandas

Pandas can clean data by:

- Deleting rows that are **not relevant**
- Removing rows with **wrong values**
- Handling **empty or NULL** values

This process is known as **data cleaning**.

Example:

If a column contains ages and some rows have missing values, Pandas can remove or fill those values to make the data usable.

Core Data Structures in Pandas

Pandas is built around **two fundamental data structures** that make data handling simple and powerful. These structures allow Pandas to work efficiently with both **simple** and **complex tabular data**.

1. Series

A **Series** is a **one-dimensional** data structure in Pandas. It is similar to a **single column in an Excel sheet** or a list with labels.

2. DataFrame

A **DataFrame** is a **two-dimensional** data structure in Pandas. It is similar to a **full Excel spreadsheet** or an **SQL table**, consisting of rows and columns.

What is a Series?

A **Pandas Series** is a **one-dimensional labeled array** capable of holding data of **any type** (integers, floats, strings, etc.).

It is similar to a **single column in a table** or an **Excel column**, but with an important feature: **labels (index)**.

A Series consists of:

- **Data values**
- **Index (labels)**

This labeling makes data access **clear and meaningful**, instead of relying only on numeric positions.

Creating a Series from a List

You can create a Series using a Python list.

If you do not provide labels, Pandas automatically assigns **default numeric indexes** starting from 0.

Example: Series of Student Scores

```
import pandas as pd
```

```
scores = [85, 90, 78, 92]
```

```
score_series = pd.Series(scores)
```

```
print(score_series)
```

Output Concept

- Index: 0, 1, 2, 3
- Values: 85, 90, 78, 92

Accessing Values Using Index

Each value in a Series can be accessed using its **index label**.

Example: Access the second score

```
print(score_series[1])
```

This returns the value at index 1.

Creating a Series with Custom Labels

You can assign **your own labels** using the index parameter.

This is useful when data needs **meaningful identification**.

Example: Series with Roll Numbers

```
import pandas as pd
```

```
marks = [76, 88, 91]
```

```
students = pd.Series(marks, index=["Roll101", "Roll102", "Roll103"])
```

```
print(students)
```

Accessing Data Using Labels

```
print(students["Roll102"])
```

This makes the code more readable and intuitive.

Creating a Series from a Dictionary (Key/Value Pairs)

A dictionary is a **natural fit** for a Pandas Series:

- **Keys** → Index labels
- **Values** → Data

Example: Temperature Readings

```
import pandas as pd
```

```
temperature = {  
    "Monday": 30,  
    "Tuesday": 32,  
    "Wednesday": 31
```

```
}
```

```
temp_series = pd.Series(temperature)
```

```
print(temp_series)
```

Important Point

- Dictionary **keys automatically become labels**
- Order follows dictionary insertion order (Python 3.7+)

Selecting Specific Items from a Dictionary

You can choose **only specific keys** using the **index** argument.

Example: Select Only Two Days

```
import pandas as pd
```

```
temperature = {  
    "Monday": 30,  
    "Tuesday": 32,  
    "Wednesday": 31  
}
```

```
temp_series = pd.Series(temperature, index=["Monday", "Wednesday"])
```

```
print(temp_series)
```

Only the specified labels are included in the Series.

Key Features of a Pandas Series

- One-dimensional data structure
- Supports **custom labels**

- Can store different data types
- Built on top of **NumPy**
- Fast and memory efficient
- Easy data access using labels

Real-Life Use Case Example

- Marks of students
- Daily temperatures
- Monthly sales
- Population of cities

Each of these can be represented naturally using a **Series**.

What is a DataFrame?

A **Pandas DataFrame** is a **two-dimensional data structure** used to store data in the form of **rows and columns**.

It is similar to:

- A **table**
- An **Excel spreadsheet**
- A **SQL table**
- A **2D array with labels**

Each column in a DataFrame is a **Series**, and each row represents a **record**.

Creating a DataFrame

A DataFrame is commonly created using a **dictionary**, where:

- Keys → Column names
- Values → Column data (lists)

Example: Employee Information

```
import pandas as pd
```

```
data = {  
    "Name": ["Alice", "Bob", "Charlie"],  
    "Salary": [50000, 60000, 55000],  
    "Department": ["HR", "IT", "Finance"]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output Concept

	Name	Salary	Department
0	Alice	50000	HR
1	Bob	60000	IT
2	Charlie	55000	Finance

Understanding Rows and Columns

- **Columns** represent attributes (Name, Salary, Department)
- **Rows** represent individual records (each employee)

This tabular structure makes DataFrames ideal for **data analysis**.

Locating Rows Using loc

Pandas provides the **loc** attribute to access one or more rows using **index labels**.

Example: Return a Single Row

```
print(df.loc[1])
```

Output Concept

- Returns data for **Bob**
- Result type: **Pandas Series**

Example: Return Multiple Rows

```
print(df.loc[[0, 2]])
```

Important Note

- Single row → **Series**
- Multiple rows → **DataFrame**

Named Indexes in DataFrame

Instead of default numeric indexes, you can assign **custom row labels** using the index parameter.

Example: Student Marks with Named Indexes

```
import pandas as pd
```

```
data = {  
    "Math": [78, 85, 92],  
    "Science": [88, 90, 95]  
}
```

```
df = pd.DataFrame(data, index=["Student1", "Student2", "Student3"])
```

```
print(df)
```

Why Named Indexes Are Useful

- Improve readability
- Make row selection more meaningful
- Useful for time-based or labeled data

Locating Rows Using Named Indexes

Once indexes are named, you can access rows using those names.

Example: Access Data for Student2

```
print(df.loc["Student2"])
```

This returns all subject marks for **Student2**.

Loading Files into a DataFrame

Pandas can load data from external files such as:

- CSV
- Excel
- JSON
- SQL databases

The most common format is **CSV (Comma-Separated Values)**.

Example: Load Sales Data from a CSV File

```
import pandas as pd
```

```
df = pd.read_csv("sales_data.csv")
```

```
print(df)
```

Advantages of Loading Files into DataFrames

- Handle large datasets easily
- Perform filtering, sorting, and analysis
- Clean missing or incorrect data

Reading CSV Files in Pandas

A **CSV (Comma-Separated Values) file** is one of the simplest and most common ways to store **large datasets**.

CSV files:

- Store data as **plain text**
- Are lightweight and easy to share
- Can be opened by **Excel, text editors, databases, and Pandas**

Because of this flexibility, CSV is a **widely used format in data science**.

Why Use CSV Files?

- Easy to create and read
- Platform-independent
- Ideal for large datasets
- Supported by almost all data tools, including **Pandas**

Loading a CSV File into a DataFrame

Pandas provides the `read_csv()` function to load a CSV file into a **DataFrame**.

Example: Load example.csv

```
import pandas as pd
```

```
df = pd.read_csv("example.csv")
```

```
print(df)
```

This reads the data from the CSV file and stores it in a **DataFrame** called `df`.

Printing the Entire DataFrame

When a DataFrame is very large, Pandas does **not display all rows by default**.

To print the **entire DataFrame**, you can use the `to_string()` method.

Example: Display Full DataFrame

```
import pandas as pd
```

```
df = pd.read_csv("example.csv")
```

```
print(df.to_string())
```

Tip

- `to_string()` is useful for **small to medium datasets**
- Avoid it for very large datasets as output may be too long

Default Display Behavior of Pandas

If you print a DataFrame **without using to_string()**:

- Pandas shows:
 - Column headers
 - First 5 rows
 - Last 5 rows

Example

```
import pandas as pd
```

```
df = pd.read_csv("example.csv")
```

```
print(df)
```

This helps keep the output **clean and readable**.

Understanding max_rows

Pandas limits how many rows are displayed using the **max_rows option**.

Check Maximum Number of Displayed Rows

```
import pandas as pd
```

```
print(pd.options.display.max_rows)
```

This shows the maximum number of rows Pandas will display before truncating the output.

Changing the Maximum Display Rows

You can increase the limit if you want to see **more rows**.

Example: Increase Maximum Rows

```
import pandas as pd
```

```
pd.options.display.max_rows = 9999
```

```
df = pd.read_csv("example.csv")
```

```
print(df)
```

Now Pandas will display more rows before truncating the output.

When to Change max_rows

- Useful during **data exploration**
- Helpful for **small datasets**
- Not recommended for extremely large datasets

Understanding Your DataFrame in Pandas

Once your **DataFrame is ready**, the next important step is **exploring and understanding the data**. Pandas provides several built-in functions that help you quickly inspect the structure, size, and summary of your dataset.

These methods are usually the **first commands** you run in any data analysis project.

1. df.head()

- Displays the **first 5 rows** of the DataFrame by default
- Helps you understand:
 - Column names
 - Type of data stored
 - Overall structure

Example

```
df.head()
```

Use case:

Check if the data loaded correctly and preview sample records.

2. df.tail()

- Displays the **last 5 rows** of the DataFrame
- Useful for:
 - Checking recent or ending records
 - Verifying data completeness at the end

Example

df.tail()

Use case:

Confirm that the dataset ends properly without missing or corrupted rows.

3. df.info()

- Provides a **summary of the DataFrame**
- Shows:
 - Number of rows and columns
 - Column names
 - Data types (int, float, object, etc.)
 - Count of **non-null values**
 - Memory usage

Example

df.info()

Why it is important:

- Helps identify **missing values**
- Useful for detecting **wrong data types**

4. df.describe()

- Generates **statistical summary** for **numeric columns**
- Includes:
 - Count

- Mean (average)
- Standard deviation
- Minimum value
- Maximum value
- Quartiles (25%, 50%, 75%)

Example

```
df.describe()
```

Use case:

Quickly understand data distribution and spot outliers.

5. df.columns

- Returns a list of **column names** in the DataFrame
- Useful when:
 - Column names are long
 - You want to loop through columns
 - You are unsure of exact column names

Example

```
df.columns
```

Output Concept:

An index-like list of column labels.

6. df.shape

- Returns a **tuple** representing:
 - Number of rows
 - Number of columns

Example

```
df.shape
```

Output Concept:

(rows, columns)

Selecting Rows & Columns in Pandas

Selecting specific rows and columns is a **core operation in data analysis**. Pandas provides multiple ways to access data depending on whether you want **columns, rows, single values, or filtered data**.

1. Selecting Columns

Columns can be selected by their **column names**.

Selecting a Single Column

- Returns a **Series**
- Useful when working with one variable

Example

```
df["Salary"]
```

Selecting Multiple Columns

- Returns a **DataFrame**
- Column names must be passed as a **list**

Example

```
df[["Name", "Department"]]
```

2. Selecting Rows by Index

Pandas provides two main methods for row selection:

.loc[] → Label-based indexing

.iloc[] → Position-based indexing

Selecting a Single Row

Using .loc[] (by label)

```
df.loc[2]
```

Using .iloc[] (by position)

```
df.iloc[2]
```

If the index is numeric, `.loc[]` and `.iloc[]` may look similar, but they work differently.

3. Selecting Specific Rows and Columns Together

You can select **both rows and columns** at the same time.

Using `.loc[]`

```
df.loc[1, "Department"]
```

Using `.iloc[]`

```
df.iloc[1, 2]
```

4. Slicing Rows and Columns

Slicing allows you to select a **range of rows and columns**.

Slice Using `.loc[]`

```
df.loc[1:3, ["Name", "Salary"]]
```

- Includes both start and end labels

Slice Using `.iloc[]`

```
df.iloc[1:4, 0:2]
```

- End index is **excluded**
- Works like normal Python slicing

5. Fast Access Using `.at[]` and `.iat[]`

These methods are optimized for **fast access to a single value**.

`.at[]` → Label-based

```
df.at[3, "Salary"]
```

`.iat[]` → Position-based

```
df.iat[3, 1]
```

When to Use

- Faster than `.loc[]` and `.iloc[]`

- Best for accessing **one specific value**

6. Filtering Rows with Conditions

Filtering helps extract **only the rows that meet certain conditions**.

Simple Condition

Example

```
df[df["Salary"] > 50000]
```

Selects employees earning more than 50,000.

Multiple Conditions (AND)

Use & and wrap each condition in parentheses.

Example

```
df[(df["Age"] > 28) & (df["Department"] == "IT")]
```

Multiple Conditions (OR)

Use | for OR conditions.

Example

```
df[(df["City"] == "Mumbai") | (df["Age"] < 25)]
```

7. Filtering Using .query()

The .query() method allows filtering using a **string expression**, making code:

- Cleaner
- More readable
- Similar to **SQL syntax**

Basic Example

```
df.query("Salary > 50000")
```

Multiple Conditions

```
df.query("Age > 30 and City == 'Mumbai'")
```

Using Variables in .query()

```
limit = 30
```

```
df.query("Age > @limit")
```

Important Rules for .query()

- Column names act like **variables**
- String values must be in **quotes**
- Use **backticks** for column names with spaces

Example

```
df.query("`Annual Salary` > 60000")
```

The **.query()** method always returns a *copy* of the DataFrame, not a view.

What Does This Mean?

When you use **.query()** to filter data:

```
filtered_df = df.query("Age > 30")
```

- filtered_df is a **new DataFrame**
- It does **not share memory** with the original df
- Changes made to filtered_df will **NOT affect** df

Example to Understand Copy vs View

```
filtered_df = df.query("Department == 'IT'")
```

```
filtered_df["Salary"] = filtered_df["Salary"] + 5000
```

The salary increase applies **only** to filtered_df
The original df remains unchanged

Comparison with Other Methods

- Boolean indexing (`df[df["Age"] > 30]`)
 - Usually returns a **copy**
- `.loc[]`
 - Can return a **view or a copy**, depending on context
- `.query()`
 - **Always returns a copy**

